**PROYECTO REFACTORIZACIÓN MVP**

**ENTREGA FINAL**

**NOMBRE MVP:**

PRÉSTAMOS TEC

**ARQUITECTURA DE SISTEMAS**

**DOCENTE:**

FERNANDO HIGUITA BENTANCUR

**INTEGRANTES:**

SANTIAGO CORTES

JUAN DAVID OSPINA

SAMUEL SILVA

**NOVIEMBRE 2026**

**UCLA**

## 1. Descripción del sistema

Préstamos TEC es una aplicación web para el punto de préstamo de apoyo tecnológico de la universidad (Bloque 1, piso 3). Reemplaza la planilla en papel donde se anotaban los préstamos, que se perdía con facilidad y tenía errores porque cada persona la llenaba a mano.

El encargado del punto usa la página para registrar usuarios y equipos, prestar y recibir equipos y consultar el inventario. El módulo que escogimos como MVP para la re-arquitectura es Inventario, porque es el que dice en qué estado está cada equipo: disponible, prestado, en mantenimiento o dado de baja. Si ese dato falla, falla todo lo demás.

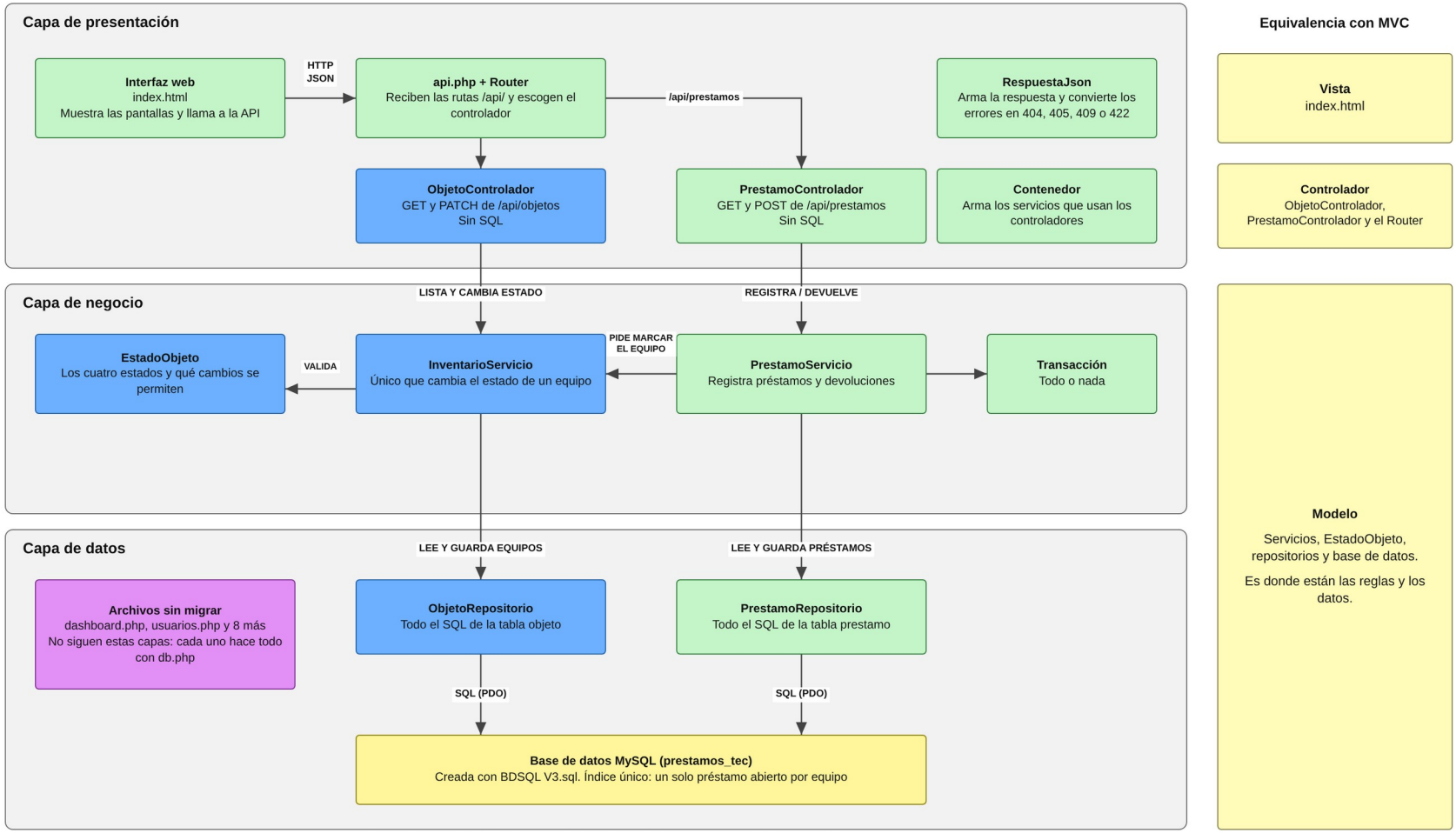
En la semana 8 priorizamos tres atributos de calidad para este módulo: mantenibilidad, confiabilidad e integridad de datos. Este documento muestra cómo quedó el sistema después de la refactorización, qué cambiamos frente a lo que habíamos planeado y cómo comprobamos que esos atributos mejoraron.

## 2. Vista lógica final del sistema

El sistema sigue siendo cliente-servidor y tiene tres partes: la interfaz web (index.html), el backend en PHP y la base de datos MySQL. Todo corre en XAMPP. Lo que cambió es cómo está organizado el backend por dentro y cómo le habla la página.

Antes, cada archivo PHP abría la conexión, escribía su SQL y armaba la respuesta. Ahora el código del MVP está separado en tres capas: presentación, negocio y datos. La figura 1 muestra cada componente en su capa y, a la derecha, cómo se ve lo mismo si se mira como MVC.

**Préstamos TEC - Vista lógica (capas y MVC) Versión 2: arquitectura final**




Las cajas azules son el módulo de Inventario (MVP). Las flechas muestran quién llama a quién.

*Figura 1. Vista lógica final*

## 2.1 Capas del backend

Esta tabla dice qué hace cada parte y en qué carpeta está:

| Capa lógica | Parte | Qué hace | Dónde está |
|---|---|---|---|
| Presentación | Entrada de la API | Recibe todas las rutas /api/ y decide qué controlador las atiende. | `.htaccess`, `src/api.php` y el Router |
| Presentación | Controladores | Leen la petición y devuelven la respuesta en JSON. No tienen SQL. | `src/Inventario/Http/`, `src/Prestamos/Http/` |
| Negocio | Servicios | Aplican las reglas del negocio y coordinan cada operación. | `InventarioServicio`, `PrestamoServicio` |
| Negocio | Dominio | Define los estados de un equipo y qué cambios se permiten. | `EstadoObjeto.php` en `src/Inventario/Dominio/` |
| Datos | Repositorios | Tienen todo el SQL de cada tabla. | `ObjetoRepositorio`, `PrestamoRepositorio` |
| Apoyo | Compartido | Están juntas en una carpeta, pero cada pieza sirve a una capa: RespuestaJson y el Contenedor a presentación, la Transacción a negocio y la Conexión a datos. | `src/Compartido/`, `src/Config/` |

Si se mira como MVC, la vista es index.html; los controladores son ObjetoControlador y PrestamoControlador, con el Router como puerta de entrada; y el modelo son los servicios, EstadoObjeto y los repositorios, que es donde están las reglas y los datos. La diferencia con un MVC clásico es que PHP no arma las pantallas: la página pide los datos en JSON a la API y ella misma los muestra.

## 2.2 API REST del MVP

Inventario y préstamos ahora se usan como una API REST. El archivo `.htaccess` de la raíz manda cualquier ruta que empiece por /api/ al archivo `src/api.php`, y el Router escoge el controlador según el método HTTP y la ruta. La dirección base en XAMPP es http://localhost/prestamos/api.

| Método | Ruta | Qué hace | Respuestas |
|---|---|---|---|
| GET | /api/objetos | Lista el inventario con el estado de cada equipo. | 200 |
| GET | /api/objetos?estado=disponible | Lista solo los equipos que se pueden prestar. | 200 |
| PATCH | /api/objetos/{id}/estado | Cambia el estado de un equipo a mantenimiento, baja o disponible. Cuerpo: {"estado": "mantenimiento"} | 200, 404, 409, 422 |
| GET | /api/prestamos | Lista los préstamos con el usuario y el equipo. | 200 |

| Método | Ruta | Qué hace | Respuestas |
|---|---|---|---|
| POST | /api/prestamos | Registra un préstamo. Cuerpo: {"id_usuario", "id_objeto", "dias", "obs"} | 201, 404, 409, 422 |
| POST | /api/prestamos/{id}/devolucion | Registra la devolución de un préstamo. Cuerpo opcional: {"obs"} | 200, 404, 409 |

Cuando algo sale mal, la API responde con un código HTTP y un JSON de la forma {"error": "mensaje"}, que la página muestra como alerta:

- **404:** el equipo, el usuario o el préstamo no existe, o la ruta no existe.
- **405:** la ruta existe pero no acepta ese método.
- **409:** la operación choca con el estado actual. Por ejemplo, el equipo ya está prestado, el préstamo ya fue devuelto o el cambio de estado no está permitido.
- **422:** faltan datos o no son válidos, como unos días de préstamo fuera del rango de 1 a 30.

La API está documentada en el README del repositorio, con un ejemplo de entrada y de respuesta para cada ruta, y en Swagger (docs/api/), donde se puede probar cada endpoint desde el navegador.

La ruta PATCH no deja poner ni quitar el estado “prestado”. Ese cambio solo ocurre al registrar o devolver un préstamo, para que nunca quede un equipo prestado sin préstamo, o al revés.

En la pantalla de Inventario, cada equipo que no está prestado tiene un selector para pasarlo a mantenimiento, darlo de baja o volverlo a disponible. Ese selector usa la ruta PATCH.

### 2.3 Lo que no se migró

Los otros diez archivos de `src/` (dashboard, usuarios, categorías, facultades, notificaciones y crear o eliminar objetos y usuarios) quedan como estaban: la página los llama directamente, siguen usando `db.php` y tienen su SQL adentro. No hacen parte del MVP.

# 3. Diagramas C4

Estos son nuestros diagramas de la semana 13, corregidos para que muestren cómo quedó el sistema al final (versión 2).

## 3.1 Contexto (nivel 1)

En este nivel no cambia nada con la refactorización: el encargado usa Préstamos TEC desde el navegador y no hay sistemas externos conectados.

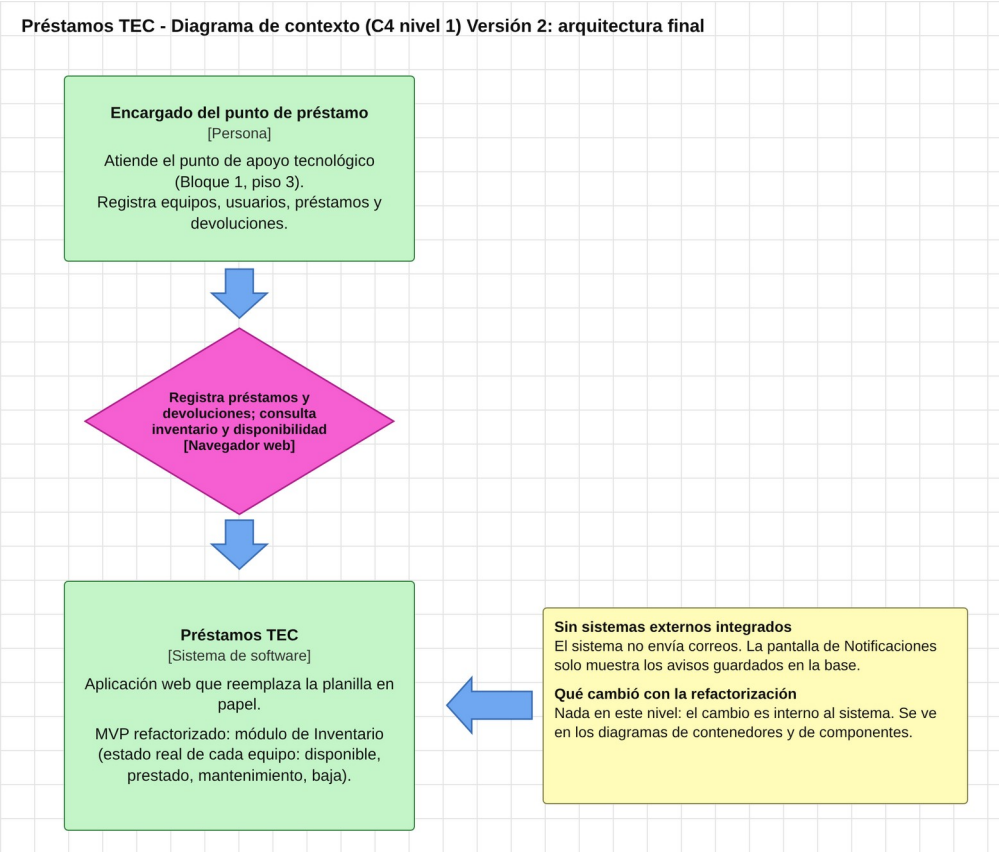



*Figura 2. Diagrama de contexto*

## 3.2 Contenedores (nivel 2)

Los contenedores son los mismos tres de antes. Lo que cambió es cómo se comunican: para inventario y préstamos la página ahora llama a la API REST, y la base de datos se creó con BDSQL V3.sql, que ya no tiene triggers ni procedimientos de préstamo.

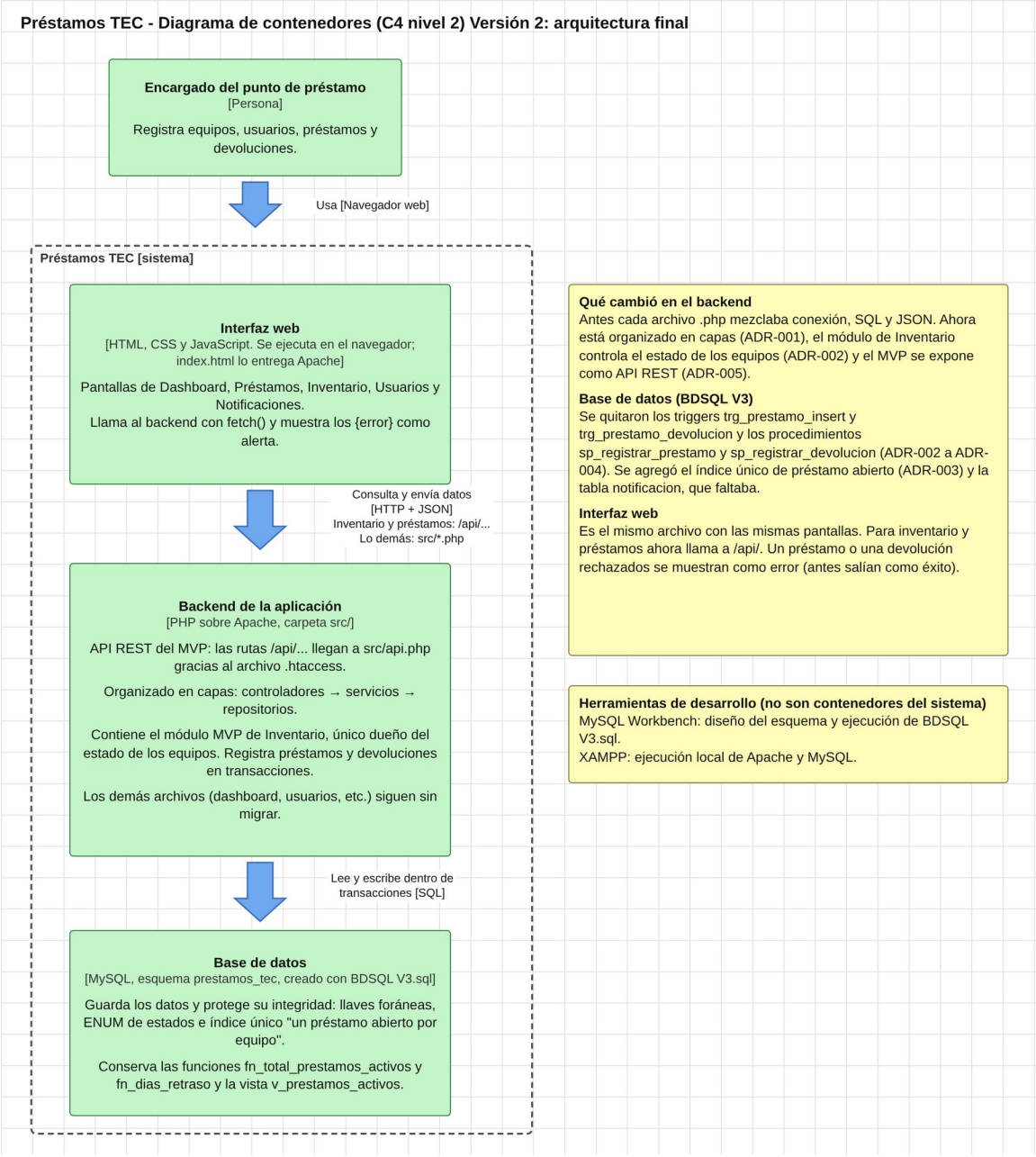



*Figura 3. Diagrama de contenedores*

## 3.3 Componentes del backend (nivel 3)

Las cajas azules son el módulo de Inventario (MVP). Inventario es el único que cambia el estado de un equipo. Cuando Préstamos necesita marcar un equipo como prestado o disponible, se lo pide a InventarioServicio. La caja morada agrupa los archivos que no se migraron.

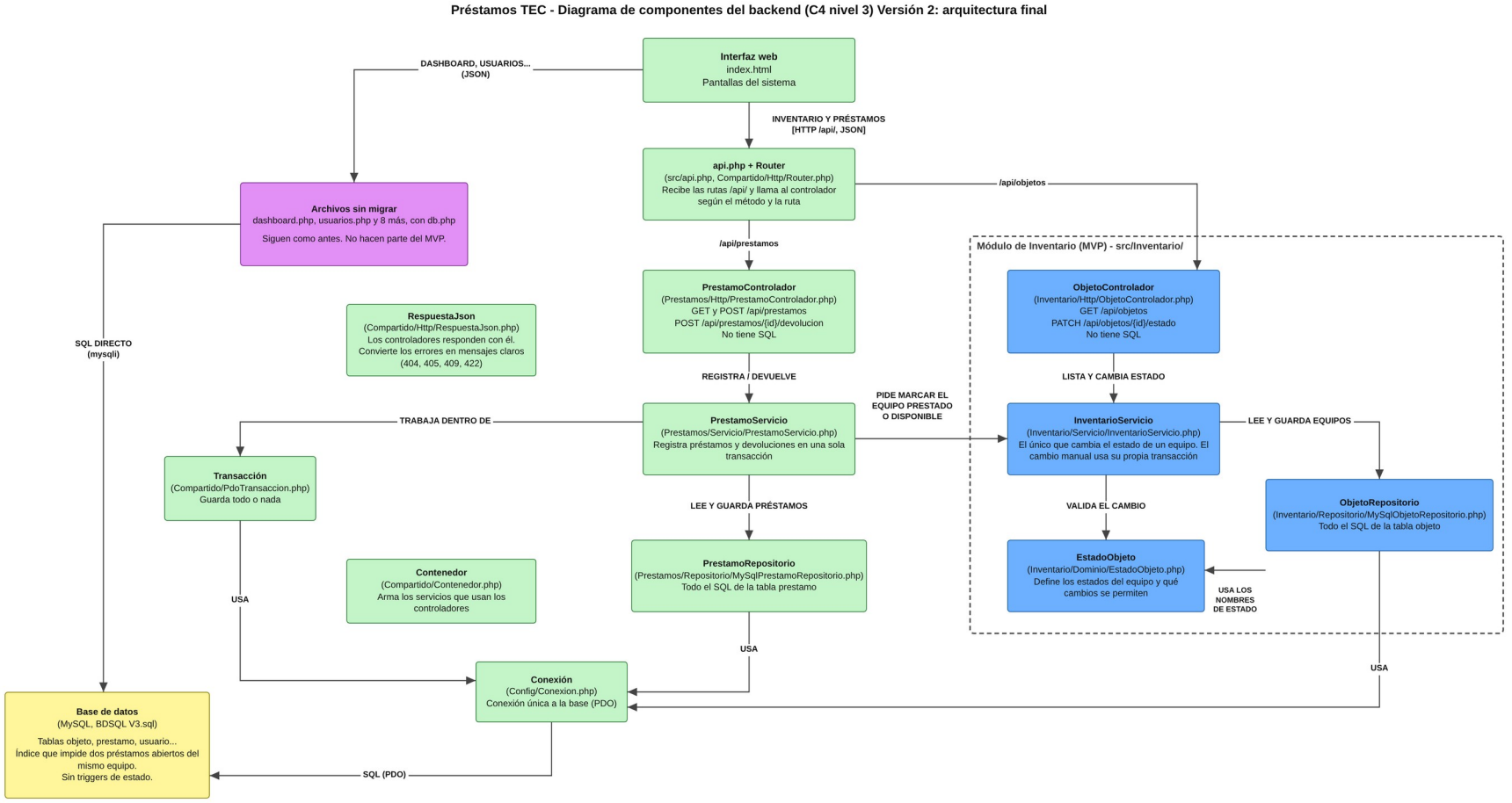



*Figura 4. Diagrama de componentes del backend*

## 4. Diagrama de despliegue

El sistema se despliega de forma local, en el mismo computador del punto de préstamo. XAMPP pone a correr Apache con PHP, que entrega la página y atiende la API, y MySQL, que guarda la base `prestamos_tec`. MySQL Workbench solo se usa para crear la base con el script; no hace parte del sistema en funcionamiento.

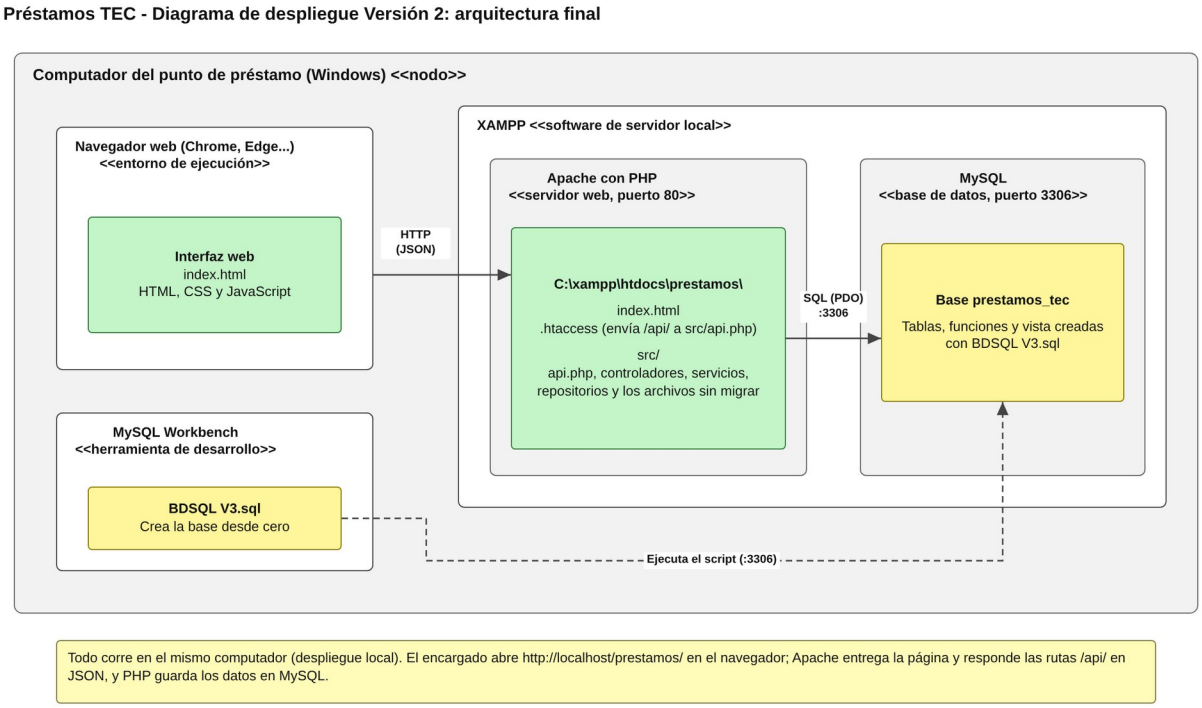



*Figura 5. Diagrama de despliegue local*

## 5. Diagrama de secuencia: registrar un préstamo

Este es el flujo principal del MVP. Lo importante está en los pasos 8 a 17: todo pasa dentro de una transacción y el equipo queda bloqueado desde el paso 11 hasta el COMMIT. Si llega otra solicitud por el mismo equipo, tiene que esperar, y cuando sigue ya encuentra el equipo prestado y recibe un 409.

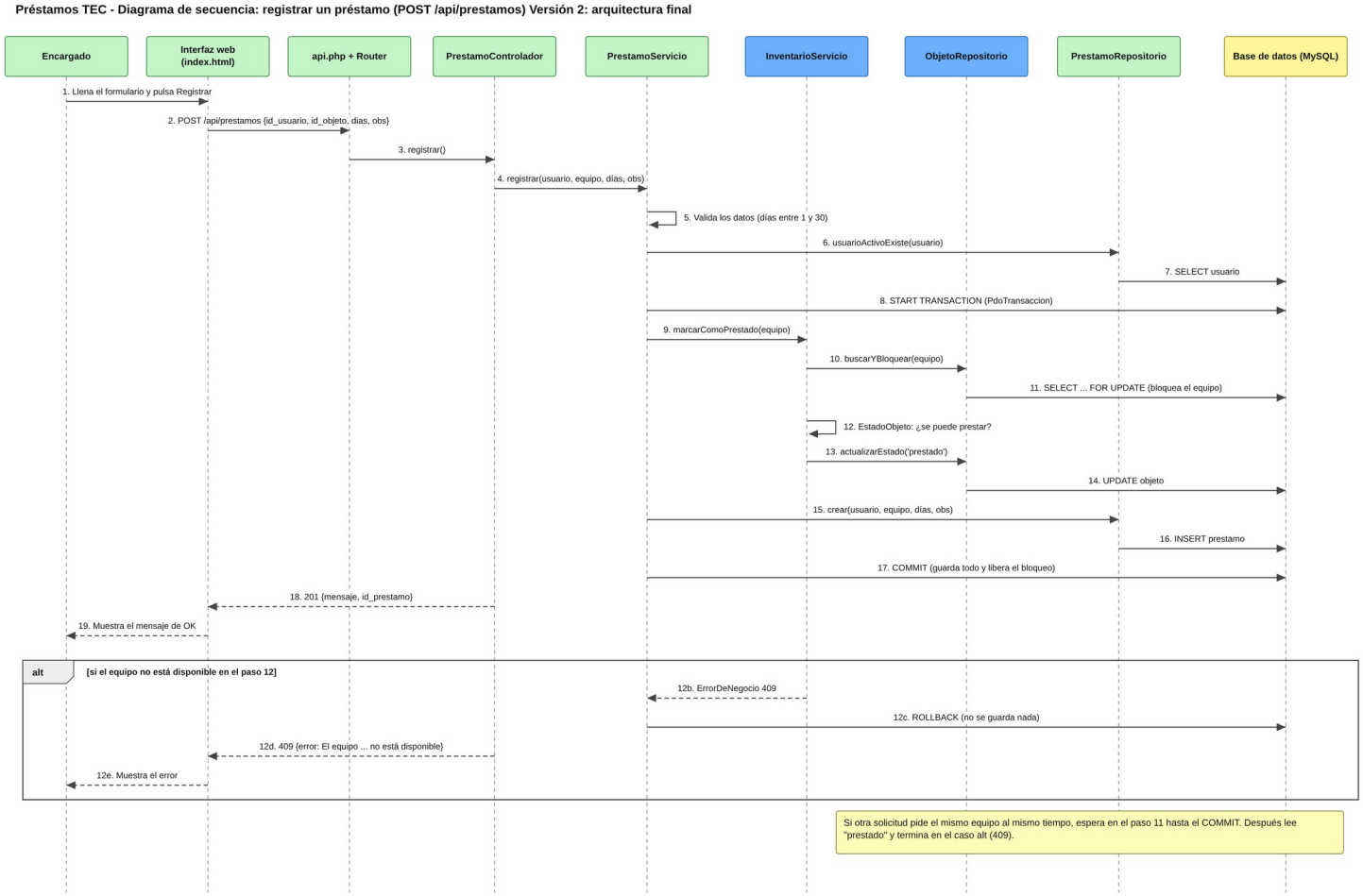



*Figura 6. Secuencia de registrar un préstamo*

# 6. Refactorización realizada

## 6.1 Problemas de la versión inicial (semana 3)

En la semana 3 identificamos tres problemas de arquitectura. En la semana 8 los buscamos en el código y encontramos la evidencia concreta. Así quedó cada uno:

| Problema (semana 3) | Lo que encontramos en el código (semana 8) | Cómo quedó |
|---|---|---|
| P1. Concentración de responsabilidades | prestamos.php abría la conexión, hacía un JOIN de tres tablas y armaba el JSON en el mismo archivo. Si el Dashboard necesitaba esos datos, había que repetir la consulta. | Resuelto en el MVP. El backend quedó en capas (ADR-001) y los controladores de la API no tienen SQL. Los diez archivos que no son del MVP siguen igual. |
| P2. Dependencia entre módulos | El estado del equipo lo cambiaban dos triggers de la tabla prestamo. sp_registrar_prestamo permitía prestar dos veces el mismo equipo y sp_registrar_devolucion respondía “OK” aunque no hubiera cambiado nada. | Resuelto. Inventario controla el estado (ADR-002), el préstamo se registra con bloqueo dentro de una transacción (ADR-003) y la devolución se verifica antes de confirmar (ADR-004). |
| P3. Escalabilidad limitada | El sistema está pensado para un solo punto de préstamo y corre en un solo computador. | No se resolvió, pero quedó una base: la API REST permite que otra pantalla u otro punto use el mismo backend. |

## 6.2 Cambios aplicados: decisiones de la semana 13 y qué cambió frente a lo planeado

En la semana 13 registramos cuatro decisiones (ADR-001 a ADR-004). Para la entrega final agregamos una quinta, ADR-005, sobre la API REST. La tabla compara lo que planeamos con lo que quedó.

| Decisión | Lo planeado | Lo que quedó | Por qué cambió |
|---|---|---|---|
| ADR-001<br>Separar el backend en capas | Migrar primero los cinco archivos de inventario y préstamos, y después los otros diez. | Los cinco archivos del MVP quedaron en capas y luego pasaron a la API REST. Los otros diez no se migraron. | Por tiempo, nos concentramos en el MVP, que es lo que pide el curso. |
| ADR-002<br>Inventario controla el estado de sus equipos | Retirar los triggers con una migración (V3__refactor_inventario.sql) y tener un script para deshacerla. | La base se reconstruyó desde cero con BDSQL V3.sql, que ya no crea los triggers ni los procedimientos de préstamo. | La base de XAMPP solo tenía datos de prueba, así que crearla de nuevo fue más simple y seguro que migrarla. |

| Decisión | Lo planeado | Lo que quedó | Por qué cambió |
|---|---|---|---|
| ADR-003<br>Registrar el préstamo con bloqueo | Una transacción con SELECT ... FOR UPDATE y un índice único en una columna generada STORED. | Igual, pero la columna objeto_en_prestamo es VIRTUAL. | MySQL no acepta una columna STORED basada en id_objeto, porque esa llave foránea tiene ON DELETE CASCADE. Con VIRTUAL el índice funciona igual. |
| ADR-004<br>Verificar la devolución | Además del cambio en el código, el paso 4 de la migración corregía los equipos que estaban “prestados” sin préstamo abierto. | El cambio en el código quedó igual. En lugar de corregir datos, se deja una consulta de control para Workbench. | Al reconstruir la base no quedaron datos viejos que corregir. |
| ADR-005<br>Exponer el MVP como API REST (nueva) | No estaba en el plan de la semana 13. | api.php con Router y .htaccess. Los cinco archivos del MVP se reemplazaron por dos controladores con las rutas de la sección 2.2. index.html ahora llama a /api/. | La entrega final pide endpoints REST. Como el backend ya estaba en capas, solo hubo que cambiar la capa de controladores. |

También hubo cambios que no estaban en ningún ADR:

- **Tabla notificacion:** el código la usaba pero nunca se había creado, por eso la pantalla de Notificaciones fallaba. Se agregó en BDSQL V3.sql.
- **Ruta de index.html:** la página llamaba a una carpeta api/ que no existía en el repositorio. En la semana 13 se cambió a src/, y ahora, con la API REST, inventario y préstamos usan /api/.
- **Pruebas automáticas:** los ADR de la semana 13 hablaban de pruebas automáticas, pero decidimos no incluirlas en la entrega. La validación se hace con los escenarios de la sección 6.4.
- **Botones de la tabla de Préstamos:** el botón de eliminar llamaba a la función sin decirle qué borrar y siempre terminaba en error, así que se quitó: un préstamo no se borra, se cierra con la devolución. Además, el botón “Devolver” ahora se desactiva mientras se procesa, para que un doble clic no envíe dos devoluciones (era la mejora pendiente del ADR-004).

## 6.3 Patrones aplicados

| Patrón | Dónde se usa | Para qué sirve aquí |
|---|---|---|
| Arquitectura en capas | Todo el código del MVP en src/: presentación, negocio y datos | Cada parte tiene una sola tarea y se puede cambiar sin tocar las demás. |

| Patrón | Dónde se usa | Para qué sirve aquí |
|---|---|---|
| Modelo-Vista-Controlador (MVC) | index.html (vista), ObjetoControlador y PrestamoControlador (controladores), servicios, dominio y repositorios (modelo) | Separa lo que se ve, lo que recibe las peticiones y lo que tiene las reglas y los datos. |
| Controlador frontal (Front Controller) | src/api.php y Router | Todas las rutas de la API entran por un solo lugar, así que los errores se manejan igual en todas. |
| Capa de servicio | InventarioServicio y PrestamoServicio | Las reglas del negocio quedan en un solo sitio y no en los controladores ni en la base de datos. |
| Repositorio | ObjetoRepositorio y PrestamoRepositorio (interfaz más la versión MySql) | Todo el SQL de una tabla está en un solo archivo. Los servicios no saben que hay MySQL detrás. |
| Inyección de dependencias | Contenedor.php | Arma cada servicio con lo que necesita, para que los controladores no tengan que hacerlo. |
| Transacción y bloqueo pesimista | PdoTransaccion y buscarYBloquear (SELECT ... FOR UPDATE) | Guarda todo o nada y no deja que dos solicitudes presten el mismo equipo a la vez. |
| Máquina de estados | EstadoObjeto | Una tabla dice a qué estados puede pasar un equipo desde su estado actual. |

## 6.4 Mejora de los atributos de calidad y validación de los escenarios

Para cada escenario de la semana 8 explicamos qué cambio lo hace posible, cómo lo comprobamos y qué obtuvimos. Las capturas están en la carpeta docs/evidencias del repositorio.

### Mantenibilidad

| Elemento | Descripción |
|---|---|
| Escenario (semana 8) | Se pide agregar un estado nuevo a los equipos, por ejemplo “reservado”. El cambio debe hacerse solo en el módulo de Inventario, sin modificar archivos de Préstamos, en menos de 4 horas. |
| Qué lo hace posible | Los estados y sus transiciones están solo en EstadoObjeto (ADR-002). El módulo de Préstamos no escribe nombres de estado: le pide a InventarioServicio que marque el equipo. |
| Cómo se valida | Revisando qué archivos habría que tocar para agregar “reservado”: la constante y sus transiciones en EstadoObjeto.php, y el valor en el ENUM de la tabla objeto (BDSQL V3.sql). |

| Elemento | Descripción |
|---|---|
| Resultado esperado | Dos cambios, EstadoObjeto.php y el ENUM de la tabla objeto, los dos de Inventario. Ninguno en src/Prestamos/. En la interfaz, el estado nuevo se mostraría con la etiqueta gris que se usa por defecto. |
| Resultado obtenido | Revisamos el código: ningún archivo de src/Prestamos/ usa los nombres de estado del equipo, y los estados con sus cambios permitidos están en una sola lista dentro de EstadoObjeto.php (captura 13). En Workbench, SHOW TRIGGERS FROM prestamos_tec no devuelve triggers (captura 08), y el cambio manual de estado desde Inventario funciona (captura 09). |

## Confiabilidad

| Elemento | Descripción |
|---|---|
| Escenario (semana 8) | Dos personas piden el mismo equipo al mismo tiempo. El sistema debe aprobar solo una solicitud y a la otra responderle claramente, en menos de 2 segundos, que el equipo ya fue prestado. Cero casos de doble préstamo. |
| Qué lo hace posible | PrestamoServicio registra el préstamo en una transacción y bloquea la fila del equipo (ADR-003). Además, el índice único uq_prestamo_abierto_por_objeto no deja guardar dos préstamos abiertos del mismo equipo. |
| Cómo se valida | 1. Abrir la página en dos pestañas, escoger el mismo equipo y registrar en las dos.<br>2. Enviar varias solicitudes POST /api/prestamos al mismo tiempo para el mismo equipo. |
| Resultado esperado | Una respuesta 201 y las demás 409 con el mensaje de que el equipo no está disponible, todas en menos de 2 segundos. En la base queda un solo préstamo abierto. |
| Resultado obtenido | En XAMPP pedimos el Bafle desde dos pestañas: la primera se registró (préstamo #4) y la segunda respondió “El equipo "Bafle" no está disponible (estado actual: prestado).” (captura 10). En las pruebas durante el desarrollo también mandamos 10 solicitudes al mismo tiempo para un mismo equipo: solo una se aprobó y en la base quedó un único préstamo abierto. |

## Integridad de datos

| Elemento | Descripción |
|---|---|
| Escenario (semana 8) | Se registra la devolución de un equipo prestado. El equipo debe pasar a “disponible” solo si existe un préstamo activo válido; si no, se rechaza. Cero equipos marcados como prestados sin un préstamo activo. |
| Qué lo hace posible | PrestamoServicio busca y bloquea el préstamo, revisa que siga abierto, comprueba que el cambio sí se guardó y solo entonces le pide a Inventario liberar el equipo, todo en una transacción (ADR-004). |

| Elemento | Descripción |
|---|---|
| Cómo se valida | 1. Devolver un préstamo y revisar que el equipo vuelva a “disponible”.<br>2. Devolver otra vez el mismo préstamo desde otra pestaña que no se haya recargado.<br>3. Ejecutar en Workbench la consulta de control del ADR-004. |
| Resultado esperado | 1: respuesta OK y el equipo disponible. 2: error “ya había sido devuelto” sin cambiar nada en la base. 3: la consulta no devuelve filas. |
| Resultado obtenido | 1: la devolución del préstamo #2 respondió OK y el micrófono volvió a disponible (capturas 03 a 05). 2: al devolver otra vez el préstamo #3 desde otra pestaña, el sistema respondió “El préstamo #3 ya había sido devuelto.” (captura 07). 3: la consulta de control no devolvió filas (captura 12). Además, un préstamo con un usuario que no existe se rechaza con 404 y no se guarda nada (captura 11). |